# 📘 Classification of Data Structures

**Data Structures** are classified into:

### 1. Linear Data Structures

- Elements are arranged sequentially.
- Types:
  - **Static** → Fixed size
    - Example: **Array**
  - **Dynamic** → Size can change
    - Examples: **List, Stack, Queue**

### 2. Non-Linear Data Structures

- Elements are arranged hierarchically.
- Types:
  - **Graph**
  - **Tree**

---

# 📘 What is a Linked List?

**Definition:**
A linked list is a **linear data structure** where:

- Elements are stored in **nodes**
- Each node contains:
  - **Data**
  - **Reference (pointer)** to the next node

---

# 📘 Types of Linked Lists

### 1. Singly Linked List

- Each node has:
  - Data
  - Pointer to next node
- Last node points to **NULL**

### 2. Doubly Linked List

- Each node has:
  - Data
  - Pointer to next node
  - Pointer to previous node

### 3. Circular Linked List

- Last node points back to **first node**
- No NULL pointer

---

# 📘 Key Characteristics

- **Dynamic Size**
  - Can grow or shrink easily
- **No Contiguous Memory**
  - Nodes are stored at different memory locations
- **Efficient Insertions & Deletions**
  - Faster than arrays (no shifting needed)

---

# 📘 Advantages and Disadvantages

### ✅ Advantages

- Dynamic size

- Efficient insertion and deletion

### ❌ Disadvantages

- Extra memory required for pointers (links)
- Sequential access (no direct/random access)

---

# 📘 Basic Operations

- **Traverse** → Moving through the linked list
- **Insert** → Adding a new node
- **Delete** → Removing a node
- **Search** → Finding a specific element

---

# 📘 Comparison with Arrays

### 🔹 Memory Allocation

- **Arrays** → Contiguous memory, fixed size
- **Linked Lists** → Non-contiguous, dynamic size

### 🔹 Insertion & Deletion

- **Arrays** → Requires shifting elements
- **Linked Lists** → Efficient (pointer updates)

### 🔹 Random Access

- **Arrays** → Direct access (O(1))
- **Linked Lists** → Sequential access (O(n))

---

# 📘 Common Use Cases

- **Dynamic Memory Allocation**
  - Useful when data size is unknown or changes frequently
- **Implementation of Other Data Structures**
  - Used in:
    - Hash Tables
    - Graphs
    - Various algorithms

---

# 📘 Applications in Computer Science

- **Implementation of Stacks and Queues**
  - Linked lists are used to build stack (LIFO) and queue (FIFO) structures efficiently.
- **Implementation of Graphs**
  - **Adjacency List Representation**
    - Most commonly used method.
    - Uses linked lists to store adjacent vertices of a graph.
- **Maintaining a Directory of Names**
  - Useful for dynamic data where entries can be added or removed easily.
- **Performing Arithmetic Operations on Long Integers**
  - Handles very large numbers that cannot fit into standard data types.
- **Manipulation of Polynomials**
  - Stores polynomial terms (coefficients and powers) in nodes of a linked list.

---

# 📘 Applications in Real World

- **Image Viewer**
  - Navigate through images using next/previous links.
- **Web Pages (Navigation)**
  - Moving between **previous and next pages**.
- **Music Player**
  - Songs are linked for forward and backward navigation.
- **GPS Navigation Systems**
  - Routes and paths can be represented using linked structures.
- **Robotics**
  - Used in path planning and dynamic data handling.
- **Task Scheduling**

- Managing tasks dynamically (adding/removing tasks efficiently).

---